SALSWIFT1（以下简称为 SF1)DSI 模块用户指南

1 术语/缩略词

FPGA：Field Programmable Gate Array	现场可编程门阵列
MIPI：Mobile Industry Processor Interface	移动产业处理器接口
DSI ：Display Serial Interface	串行显示接口
DCS ：Display Command Set	显示控制命令集
DPI ：Display Pixel Interface	并行显示接口
aIDPI ：Anlogic Special Display Pixel Interface	自定义并行显示接口

2 简介

SF1 系列 FPGA 集成了 2 组 MIPI DSI Controller 与 MIPI D-PHY 模块，由于 DSI Controller 与 MIPI D-PHY 不能分开使用，下文将 DSI Controller 与 MIPI D-PHY 的组合，简称为 DSI 模块。每组 DSI 模块最大支持 4Lane 模式，可配置为独立的主设备或从设备使用，也可以将 2 组 DSI 合并，拓展成最大 8Lane 的输入输出接口。

DSI 模块具有如下性能特征：

- 支持 MIPI 联盟 DCS 规范中的所有命令
- 每一个 D-PHY 通道支持高达 2.2Gbps 的数据速率。
- D-PHY 的数据通道 0 支持双向通信和 Escape Mode。
- 支持连续和非连续发包。
- 支持未用 PLL 的超低功耗模式（Ultra Low-Power Mode)。
- 支持 ECC and Checksum 功能。
- 支持发送结束传输包(EoTp)。
- 故障恢复策略。
- 丰富的接口选择：
  - 用于 DCS 命令的 APB 接口。
  - 用于 MIPI Video Mode 的 DPI 接口。
  - 用于 Command Mode 的 aIDPI 接口。
- DPI 接口信号极性可编程。
- 增强型 DPI(aIDPI)接口，额外支持 WMS 和 WMC 指令。
- 支持 DSI 协议下的自定义数据组包。
- 支持测试模式，可独立生成测试画面。
  - 没有 DPI 协议的横向和竖向彩条。
  - 没有 DPI 协议的灰阶画面。

2.1 接口与功能描述

DSI 模块包含物理层硬核与上层控制逻辑，结构框图如下所示。其中 DPI 接口与 aIDPI 接口端口复用，用于 HOST 模式，发送 Video Mode 或 Command Mode 的高速图像、视频数据，RAWDATA 接口用于 SLAVE 模式下 DSI 接收数据的访问。APB 接口用于 DSI 模块本身的配置与状态回读，也可作为 DPI 接口、RAW DATA 接口的补充，收发 DCS 数据。

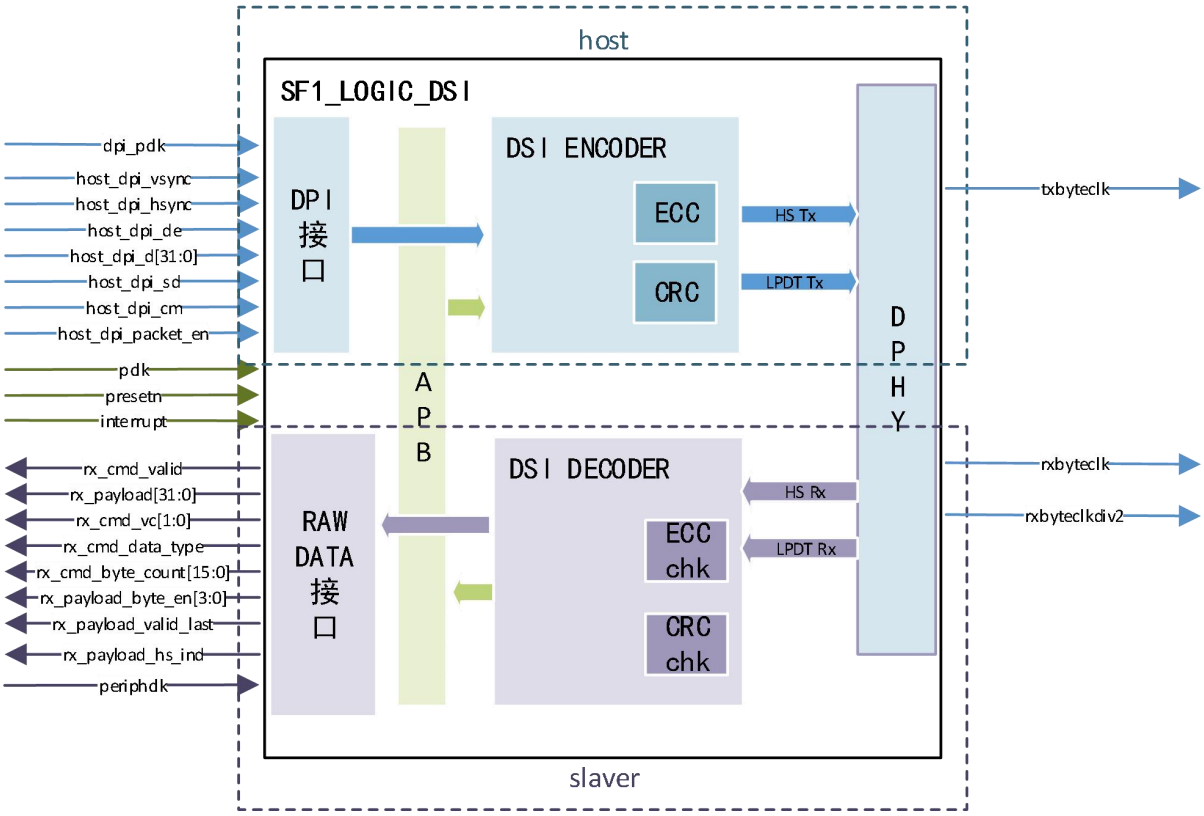

图 2-1 MIPI DSI Controller 框图

DSI 模块在使用前，需要将 DSI 控制器配置在相应的工作模式。DSI 配置链路如下图所示，以 APB 串联的方式实现。通过 soft_ip_apbm_en 的切换，可以完全由 MCU 控制，作为一个标准的 MCU 外设来配置使用，也可以在逻辑侧搭建一个 apb master 功能，由逻辑控制访问。DSI 模块分别位于 APB 配置链路的第一级和第三级，DSI1 对应的位置约束为 x18y0z0，对应的 APB 基地址为 0xe0090000，DSI0 对应的位置约束为 x0y18z0，对应的 APB 基地址为 0xe0080000。DSI 偏移地址及其含义可参照 DSI 寄存器表格。

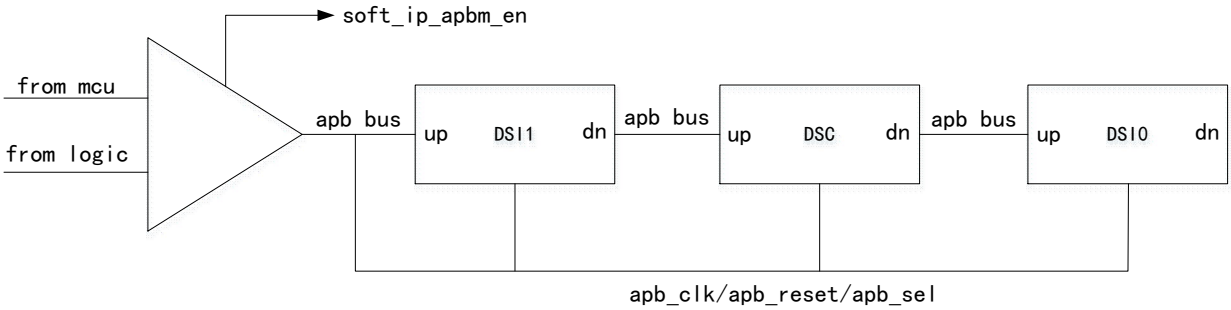

图 2-2 APB 配置级联拓扑结构

DSI 模块的配置接口满足标准的 APB 接口时序，信号定义如下表所示。其中 HARDWIRE 代表信号为固定的硬件连接关系，只能连接到上图 APB 串联链路中上/下一级 APB 接口的输入输出。配置链路的起点为 MCU

模块，所以无论是否使用 MCU 进行 DSI 配置，均需要例化 MCU 模块，借用 MCU 模块的 APB 数据通路。标有 HRADWIRE 的信号只用于帮助用户进行模块仿真，用户无法改变布线，也不能通过在线调试工具抓取信号。

DSI 模块 APB 接口信号定义如下：

表 2-1 DSI 配置接口信号列表

名称	方向	位宽	接口说明
preset	输入	1	APB 接口复位信号；高电平有效
apb_pclk	输入	1	APB 接口时钟信号，连接 MCU 模块的输出
apb_paddr_up	输入	12	配置链路中 DSI 配置入口，HARDWIRE
apb_pprot_up	输入	3	配置链路中 DSI 配置入口，HARDWIRE
apb_pstrobe_up	输入	4	配置链路中 DSI 配置入口，HARDWIRE
apb_pwdata_up	输入	32	配置链路中 DSI 配置入口，HARDWIRE
apb_prdata_up	输出	32	配置链路中 DSI 配置入口，HARDWIRE
apb_pready_up	输出	1	配置链路中 DSI 配置入口，HARDWIRE
apb_pslverr_up	输出	1	配置链路中 DSI 配置入口，HARDWIRE
apb_pwrite_up	输入	1	配置链路中 DSI 配置入口，HARDWIRE
apb_psel_up	输入	1	配置链路中 DSI 配置入口，HARDWIRE
apb_dMode_up	输入	1	配置链路中 DSI 配置入口，HARDWIRE
apb_penable_up	输入	1	配置链路中 DSI 配置入口，HARDWIRE
apb_paddr_down	输出	12	配置链路中 DSI 配置出口，HARDWIRE
apb_pprot_down	输出	3	配置链路中 DSI 配置出口，HARDWIRE
apb_pstrobe_down	输出	4	配置链路中 DSI 配置出口，HARDWIRE
apb_pwdata_down	输出	32	配置链路中 DSI 配置出口，HARDWIRE
apb_prdata_down	输入	32	配置链路中 DSI 配置出口，HARDWIRE
apb_pready_down	输入	1	配置链路中 DSI 配置出口，HARDWIRE
apb_pslverr_down	输入	1	配置链路中 DSI 配置出口，HARDWIRE
apb_pwrite_down	输出	1	配置链路中 DSI 配置出口，HARDWIRE
apb_dMode_down	输出	1	配置链路中 DSI 配置出口，HARDWIRE
apb_penable_down	输出	1	配置链路中 DSI 配置出口，HARDWIRE

DSI 控制器的数据输入输出接口定义如下

表 2-2 DSI 数据接口信号列表

名称	方向	位宽	接口说明
interrupt	输出	1	DSI 错误中断信号；高电平有效
clkesc	输出	1	接收时的 LP 模式采样时钟 发送时 LP 模式的驱动时钟
osc_clk_in	输入	1	DSI 模块时钟源，固定连接到 OSC 的输出
txbyteclk	输出	1	来源于 DPHY Tx，MIPI 线时钟的四分之一
host_dpi_pclk	输入	1	DPI/aIDPI 接口时钟
host_dpi_vsync	输入	1	DPI 接口场同步信号 aIDPI 接口写缓存开始信号
host_dpi_hsync	输入	1	DPI 接口行同步信号
host_dpi_de	输入	1	DPI/aIDP 接口数据有效信号
host_dpi_d	输入	32	DPI/aIDPI 接口数据
host_dpi_sd	输入	1	视频传输终止控制信号
host_dpi_cm	输入	1	切换颜色模式控制信号
host_dpi_packet_en	输入	1	特殊包使能信号
rxbyteclk	输出	1	来源于 DPHY Rx，MIPI 线时钟的四分之一
rxbyteclkdvi2	输出	1	来源于 DPHY Rx，MIPI 线时钟的八分之一
rx_periphclk	输入	1	接收端口同步时钟
rx_cmd_valid	输出	1	数据包的包头有效信号
rx_cmd_vc	输出	2	数据包虚拟通道号
rx_cmd_data_type	输出	6	数据包类型
rx_cmd_byte_count	输出	16	短包时为数据内容，长包时为数据长度
rx_payload	输出	64	接收数据
rx_payload_valid	输出	1	接收数据有效信号
rx_payload_byte_en	输出	4	接收数据字节使能
rx_payload_valid_last	输出	1	数据包的最后一个数据指示信号
rx_payload_hs_ind	输出	1	数据包类型指示 1’b1：HS 包；1’b0：LP 包
cn	双向	1	HARD WIRE
cp	双向	1	HARD WIRE

dn0	双向	1	HARD WIRE
dp0	双向	1	HARD WIRE
dn1	双向	1	HARD WIRE
dp1	双向	1	HARD WIRE
dn2	双向	1	HARD WIRE
dp2	双向	1	HARD WIRE
dn3	双向	1	HARD WIRE
dp3	双向	1	HARD WIRE

2.2 功能仿真

DSI 模块带有仿真模型用于支持客户仿真，无特殊使用要求，仿真时需要注意：

用户手写激励源时，需注意 weak1/weak0 与 strong1/strong0 的区别，以标识 MIPIIO 的 LP 与 HS 电平。

对于 APB 配置链路上最后一级的 apb_prdata_down、apb_pready_down、apb_pslverr_down 这些输入信号，需要进行赋值操作，否者会因为输入信号缺失，引入系统信号的未知状态。推荐处理方式如下：

```
...
.apb_dMode_down          (),
.apb_paddr_down          (),
.apb_penable_down        (),
.apb_pprot_down          (),
.apb_prdata_down         (32'd0),
.apb_pready_down         (1'b0),
.apb_pslverr_down        (1'b0),
.apb_pstrobe_down        (),
.apb_pwdata_down         (),
.apb_pwrite_down         (),
...
```

3 DSI 模块作为 SLAVE 使用

3.1 IP 调用

TD 软件支持通过 IPGUI 的方式调用 DSI 模块，界面如下：

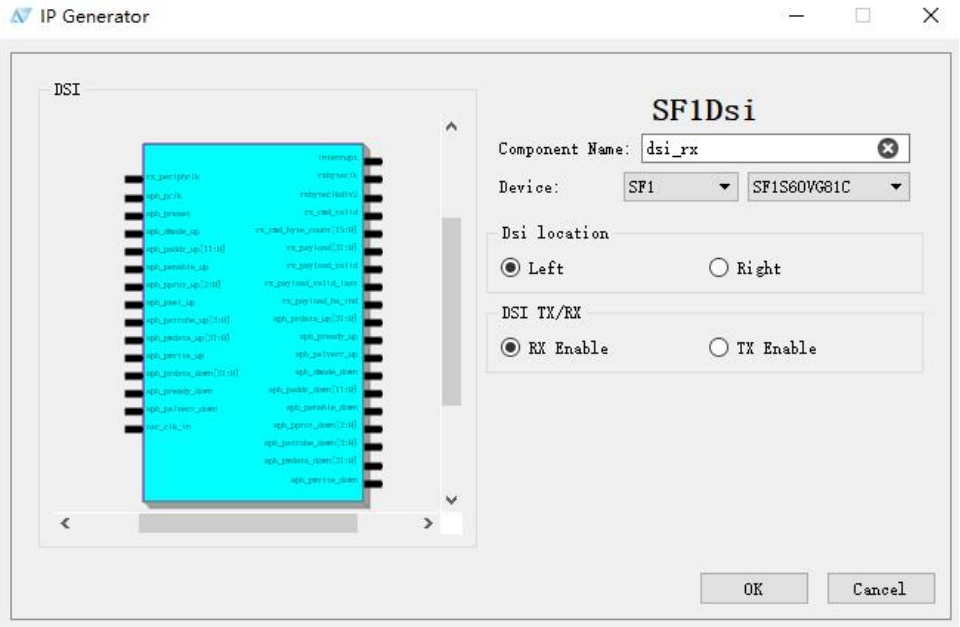

图 3-1 DSI RX 模块框图

IPGUI 可配置选项说明如下：

表 3-1 DSI IPGUI 参数说明

参数	说明
Dsi location	DSI 位置约束 Left 对应 DSI0，坐标 x0y18z0，Right 对应 DSI1，坐标 x24y0z0。 选中后，软件会自动添加相应的位置约束，需要与 PCB 对应
DSI TX/RX	DSI 模式选择 RX Enable 对应 SLAVE 模式，TX Enable 对应 HOST 模式

对于 DSI 模块熟悉的用户，也可以不经过 IPGUI 方式，手动调用 DSI 模块，此时软件无法获知用户实际操作的是两个 DSI 模块中的哪一个，不会自动生成 DSI 位置约束文件，需要用户手动添加位置约束。参考示例：

```
set_inst_assignment      { dsi0_inst_name }    { LOCATION = X0Y18Z0; }
set_inst_assignment      { dsi1_inst_name }    { LOCATION = X24Y0Z0; }
```

3.2 寄存器配置

当 DSI 模块作为 SLAVE 使用时，需要配置 D-PHY 并设置 DSI 模块为 SLAVE 模式，寄存器配置如下：

- 配置 DSI/D-PHY

  配置 CLKMGR_CFG(0x4)寄存器，设置 Time Out 时钟与 TX Escape 时钟的分频系数；

  配置 PHY_DIG_CTRL0(0x130)寄存器，使能数据对齐模块，PHY 上电；

  配置 PHY_DIG_TO_0(0x148)寄存器，设置 DSI 标准中 tInit/tLpx/tClkZero 的值，以时钟 clk_txesc 的周期为单位时间；

  配置 PHY_DIG_TO_1(0x14C) 寄存器，设置 DSI 标准中 tWakeup/tClktrail/tClkSettle 的值，以时钟 clk_txesc 的周期为单位时间；

  配置 PHY_DIG_TO_2(0x150)寄存器，设置 DSI 标准中 tHstrail/tHsprep/tHTasure 的值，以时钟 clk_txesc 的周期为单位时间；

  设置 PHY_DIG_TO_3(0x154) 寄存器，设置 DSI 标准中 tHsexit/tClkpost/tHssettle/tHszero 的值，以时钟 clk_txesc 的周期为单位时间；

- 复位释放/上电以及 DSI 模式选择

  配置 PHY_RSTZ(0xB0) 寄存器的 bit[2:0] 为 3’b111 ，把 D-PHY 从 powerdown/reset 的状态释放，并使能 D-PHY clock Lane；

  配置 PWR_UP(0x0) 寄存器的 bit0 为 1’b1，让 DSI Core 上电；同时设置 bit4 选择 DSI 为 Slave 工作模式。

- 可选配置内容

对于电路板引入的时钟线和数据线之间的偏差、数据线与数据线之间的偏差，DSI 协议引入了一种 skew 自校准的机制，即在线速率高于 1.5Gbps 时，HOST 可选择发送特定校准序列，让 SLAVE 动态调整。SF1 的 DSI 模块也支持这个功能，对于 Skew 较大的硬件电路，可以打开 PHY_DIG_CTRL0 寄存器的 deskew sync patten detect 功能，以获得更好的接收性能，由于该过程不是强制要求，是否生效要决于发送端是否有发校准序列，必要时，也可以通过 PHY_DIG_CTRL1 与 PHY_DIG_CTRL2 寄存器手动调节，获得最佳的采样位置。

3.3 DSI 接收数据

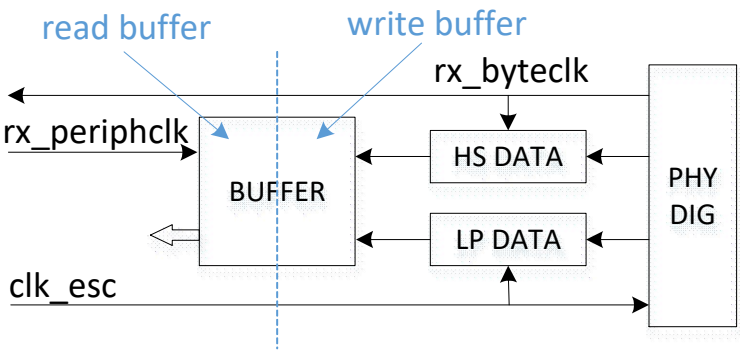

图 3-2 SLAVE 模式数据流向简图

DSI 模块作为 SLAVE 使用时，时钟与数据的驱动关系如上图所示。DSI 模块采用 MIPI 随路时钟分频后的 rx_byteclk 采样与驱动 MIPI HS 数据，以外部输入 clk_esc 分频后的时钟，采样与驱动 MIPI LP 数据，然后将 HS 数据与 LP 数据写入同一个 DATA BUFFER 中，最后由用户提供的 rx_periphclk 读出使用。作为读

时钟的 rx_periphclk，可以是 MIPI 随路时钟分频后的 rxbyteclk，也可以是 PLL 产生的全局时钟资源。需要注意的是，如果采用 rxbyteclk 作为数据读出时钟 rx_periphclk，且 mipi 时钟线存在 LP 与 HS 模式动态切换，这个时钟将将是断续的，会导致 LP 数据无法及时读出。

对于连续的数据流，一般要求数据读出时钟 rx_periphclk，等于或略大于 MIPI 随路时钟的 4 分频，以保证数据 Buffer 不会溢出。数据读出时，由 rx_payload_hs_ind 信号，标识当前读出的数据包是 HS 数据包还是 LP 数据包，同时会按照 DSI 协议校验数据包的 ECC 和 CRC，如果存在误码，数据包会被 DSI 模块丢掉。

3.4 SLAVE 模式的 BTA 处理

DSI 模块在 SLAVE 模式下通道 0 的双向通信是通过 APB 写数据的方式实现的，具体操作如下：

用户根据包类型，检测出数据包中的 DCS 读命令，通过 GEN_PLD_DATA（0x5C）寄存器依次写入要回复的数据内容，DSI 模块会以写入的第一个字节为数据包类型，判断是发送短包，还是以第二、三字节为数据长度发送长包，用户写入数据时需要预留出 ECC 与 CRC 位置，供 DSI 模块填充使用。物理层的 BTA 切换序列由 DSI 模块自动完成，不需要用户参与。

用户通过 GEN_PLD_DATA（0x5C）寄存器写入的发送 Buffer 深度为 64byte，以 apb 接口 32 位并行写入，MIPI LP 串行的方式读出，写速率一般会高于发送速率，当数据包长度超过最大 Buffer 深度时，会有数据溢出风险，此时需要通过 CMD_PKT_STATUS 寄存器，关注 Buffer 满状态，避免在 Buffer 满时仍在写数据。

4 DSI 作为 MASTER 使用

4.1 IP 调用

TD 软件支持通过 IPGUI 的方式调用 DSI 模块，界面如下：

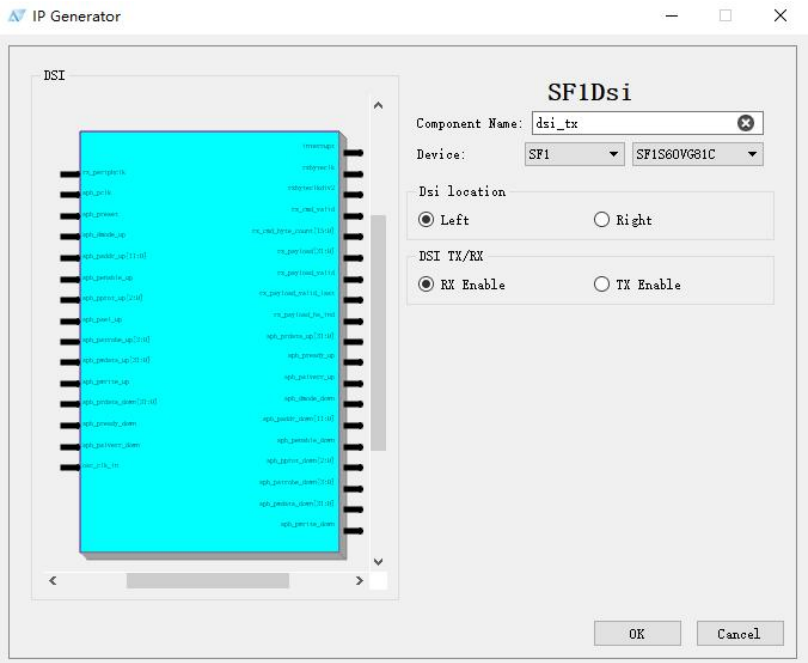

图 4-1 DSI TX 模块框图

IPGUI 可配置选项说明如下：

表 4-1 DSI IPGUI 参数说明

参数	说明

Dsi location	DSI 位置约束 Left 对应 DSI0，坐标 x0y18z0，Right 对应 DSI1，坐标 x24y0z0。 选中后，软件会自动添加相应的位置约束，需要与 PCB 对应
DSI TX/RX	DSI 模式选择 RX Enable 对应 slave 模式，TX Enable 对应 HOST 模式

对于 DSI 模块熟悉的用户，也可以不经过 IPGUI 方式，手动调用 DSI 模块，此时软件无法获知用户实际操作的是两个 DSI 模块中的哪一个，不会自动生成 DSI 位置约束文件，需要用户手动添加位置约束。参考示例：

```
set_inst_assignment      { dsi0_inst_name }    { LOCATION = X0Y18Z0; }
set_inst_assignment      { dsi1_inst_name }    { LOCATION = X24Y0Z0; }
```

4.2 DSI Host 寄存器配置

DSI 模块作为 HOST 时，支持 Command Mode 和 Video Mode。无论使用哪种发送模式，都需要通过 APB 接口对 DSI 模块本身进行相关配置。Command Mode 和 Video Mode 通用配置部分如下：

- 配置 DSI/D-PHY

  配置 CLKMGR_CFG(0x4)寄存器，设置 Time Out 时钟与 TX Escape 时钟的分频系数；

  配置 PHY_IF_CFG(0xB4)寄存器，选择有效的 Line 数，如 4line/2line/1line；

  配置 PCKHDL_CFG(0x18)寄存器，使能 EoTp 和 BTA，非必需；

  配置 TX_PLL(0x12C)寄存器，配置 DSI 专用 PLL 的参考时钟分频系数和反馈时钟分频系数，同时选择 PLL 的参考时钟源；

  配置 PHY_DIG_CTRL0(0x130)寄存器，设置 zone 模式为 1x/2x/4x/8x/16x，以及时钟相位悬着，D-PHY 上电工作；

  配置 PHY_DIG_TO_0(0x148)寄存器，以时钟 clk_txesc 的周期为单位时间，设置 DSI 标准中 tInit/tLpx/tClkZero 的值；

  配置 PHY_DIG_TO_1(0x14C) 寄存器，以时钟 clk_txesc 的周期为单位时间，设置 DSI 标准中 tWakeup/tClktrail/tClkSettle 的值；

  配置 PHY_DIG_TO_2(0x150)寄存器以时钟 clk_txesc 的周期为单位时间，设置 DSI 标准中 tHstrail/tHsprep/tHTasure 的值，；

  设置 PHY_DIG_TO_3(0x154) 寄存器，以时钟 clk_txesc 的周期为单位时间，设置 DSI 标准中 tHsexit/tClkpost/tHssettle/tHszero 的值；

- 配置 Command Mode/Video Mode

  配置 MODE_CFG(0x20)寄存器，选择 DSI 模块工作在命令模式或视频模式。

- 等待 PLL 锁定

  读寄存器 PHY_STATUS(0xC0)的 PLL LOCK 标志，等待 PLL 锁定。

- 复位释放/上电与 HOST 模式选择

  配置 PHY_RSTZ(0xB0)寄存器的 bit[2:0]，把 D-PHY 从掉电与复位状态释放，使能 D-PHY clock

Lane；

配置 PWR_UP(0x0)寄存器，给 DSI CORE 上电，同时选择为 Host 工作模式。

配置 LPCLK_CTRL(0xA0) 寄存器的 bit[0]，使能时钟。

至此，DSI 模块作为 HOST 使用的通用配置部分，配置完成。如果 auto_clkLane_ctrl 未使能，则可以看到 MIPI 时钟线有连续的 HS 时钟输出。auto_clkLane_ctrl 使能后，在没有数据发送时，时钟线会退回 LP 模式，直到 DPI/aIDPI 写入数据，重新切回 HS 模式。

4.3 DSI Host 线速率计算

为了保证时钟性能，要求 PLL 输出频率在 600MHz 以上。考虑到低比特率的 MIPI 传输场景，DSI 模块引入了 zone 的概念，由 PHY_DIG_CTRL0 寄存器进行配置。nx zone 表示 tx 输出 nbit，为 1 个 UI 数据。例如，PLL 产生时钟为 600MHz，1x zone 时，tx 每输出 1bit，表示 1 个 UI 数据，对应的线速率就是 1.2Gbps；2x zone 时，tx 每输出 2 bits，表示 1 个 UI 数据，对应的线速率就是 600Mbps。以此类推，MIPI 线速率的计算方法如下所示：

$$\text{MIPI 线速率} = \frac{\text{refclk}\times\text{fb_div}}{\text{pre_div}\times\text{zone}} \qquad \text{公式 4-1}$$

考虑到 DSI 实际传输带宽的效率，DSI 的理论带宽一般应大于等于 1.1 倍的 dpi 接口带宽（多余带宽用 null 包或 LP11 填充），即：

$$\frac{\text{Lane}\times 2\times\text{refclk}\times\text{fb_div}}{\text{pre_div}\times\text{zone}} \geq 1.1\times\text{dpiclk}\times\text{dpi_DeepColor} \qquad \text{公式 4-2}$$

注：

dpiclk：为 dpi 接口同步时钟。

dpi_DeepColor：为 dpi 接口数据颜色深度。

Lane：DSI 配置的有效 Lane 对数。

refclk：PLL 选择的参考时钟频率。

fb_div：寄存器 TX_PLL 中设置的反馈时钟分频系数。

pre_div：寄存器 TX_PLL 中设置的参考时钟分频系数。

4.4 Video Mode 发送数据

Video Mode 发送数据时，除了上述配置寄存器配置外，还需配置如下寄存器。下面寄存器可以在等待 PLL 锁定前进行配置，也可以在上述配置 DSI Host 流程完成后进行配置，其配置如下：

- 视频模式(Video Mode)选择

  配置 DPI_COLOR_CODING（0XC） 寄存器，选择 color Mode ，如 24bit RBG, 24bit YCbCr ，DSC24 compressed data；

  配置 VID_MODE_CFG(0x24) 寄存器，选择视频帧传输方式,如突发模式(Burst Mode)和非突发方式的同步脉冲模式(sync_pulse Mode) 或同步事件模(sync_event Video Mode)，以及设置 DSI 在视频消隐期间是否进入 LP 模式。 该寄存器同时也用于选择视频源为 DPI 接口还是测试画面生成器以及测试画面格式；

  配置 PHY_TMR_LPCLK_CFG(0xA4)/PHY_TMR_CFG(0xA8)寄存器，设置 phy_clklp2hs/phy_clkhs2lp timing window and phy_lp2hs/phy_hs2lp 参数值，从而设定 DSI Clock Lane 和 Data Lane LP 到 HS，HS 到 LP 状态转换需要的时间；

配置 MODE_CFG(0x20)寄存器 bit[0]，选择 DSI 工作在 Video Mode；

- DSI 数据包及视频图像分辨率设置

配置 VID_HLINE_TIME(0x3C)寄存器，设置一行数据的总时间，需映射到 DSI 的 Lanebyte clock 的周期数；

配置 VID_PKT_SIZE(0x28)寄存器，设置一个数据包的像素大小(一行的总像素/VID_NUM_CHUNKS)；

配置 VID_NUM_CHUNKS(0x2C) 寄存器，设置视频一行被分成多少个数据块（chunks），如果设置为 0 或者 1，即一行像素为一个数据包发送，只是设置为 1 时可以跟空数据包。如果是在突发模(Burst Mode)下，DSI 将忽略该寄存器设置，并将一行数据以一个数据包发送；

配置 VID_NULL_SIZE(0x30) 寄存器，设置在一个数据块包发送后，将插入的空包个数；

配置 VID_HSA_TIME(0x34)寄存器，设置 HSA 的大小，需映射到 DSI 的 Lanebyte clock 的周期数；

配置 VID_HBP_TIME(0x38) 寄存器，设置 HBP 的大小，需映射到 DSI 的 Lanebyte clock 的周期数；

配置 VID_VSA_TIME(0x40) 寄存器，设置 VSA 的大小；

配置 VID_VBP_LINES(0x44 寄存器，设置 VBP 的大小；

配置 VID_VFP_LINES(0x48) 寄存器，设置 VFP 的大小；

配置 VID_VACTIVE_LINES(0x4C) 寄存器，设置 VACTIVE 的大小；

当以上所有寄存器都配置完成后，DPI 总线接口数据便能够通过 DSI 视频模式的方式发送出去。需要注意的是，当 DSI 选择 DPI 接口用于视频数据流时，DSI 将使用检测到的第一个 Vsync，作为一帧视频数据的开始。在这之前的所有数据流将被忽略。

在 Video Mode 中，MIPI DSI 控制器需要知道时钟通道从高速到低功耗以及从低功耗到高速所需要的时间，使用内部参数与 PHY 层计时配置，来确定是否有时间让时钟线进入低功耗模式。

DSI 模块数据 Buffer 大小为 1440x32bits，假设为 RGB888 模式，为了避免 FIFO 读空和溢出，一个数据包的最大个数应不超过 1920 个像素。视频的突发模式与非突发模式会略有区别，如下所示：

- Burst Mode

在该模式下，整行像素数据被缓冲到 Buffer 中，并在没有中断的单个数据包中传输。这种传输模式要求 DPI 像素 Buffer 有能力在其内部存储一整行有效像素数据。当所需像素带宽和 DSI 链路带宽之间的差异较大时，推荐使用这种模式。这使得 MIPI DSI Controller 能够在一次数据突发中。快速地传输完整行有效视频数据，然后进入到低功耗模式。其传输示意图如下图所示：

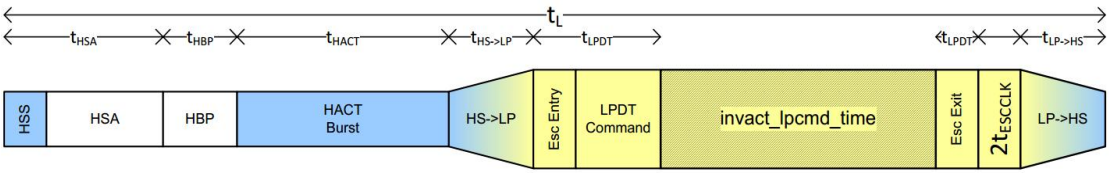

图 4-2 Burst Mode 传输

- Nor-Burst Mode

在该模式下，分为非突发模式同步脉冲（Nor-Burst Mode with sync pulse）传输和非突发模式同步事件（Nor-Burst Mode with sync events）传输。需要将一行有效视频数据，划分为多个 DSI 包进行传输，这样做是为了匹配像素所需的带宽与 DSI 链路带宽。在这种模式下，控制器配置不需要一整行像素数据存储在 DPI 像素的 Buffer 中。其传输示意图如下图所示：

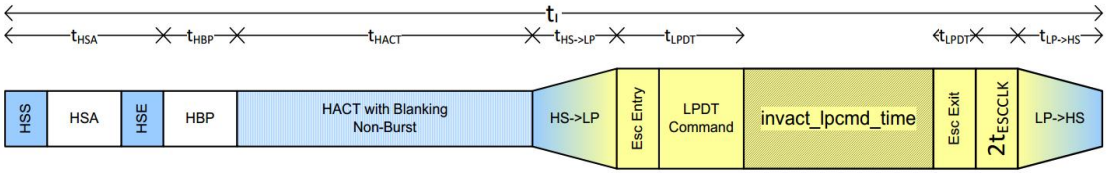

图 4-3 Nor-Burst Mode with sync pulse 传输

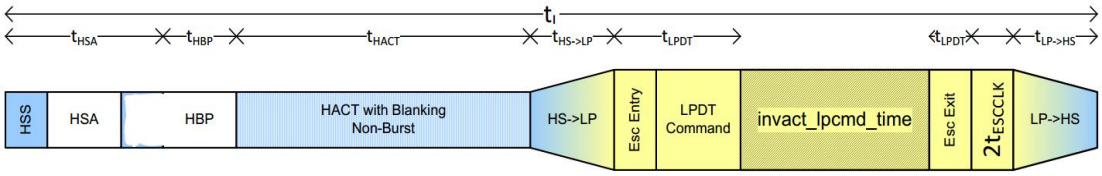

图 4-4 Nor-Burst Mode with sync events 传输

■ 视频模式传输选择

Burst Mode 和 Nor-Burst Mode 的选择主要取决于系统配置和设备需求，需要根据应用场景选择视频传输方式。

Burst Mode 更有利，因为它增加了链路在低功耗模式下花费更多时间的概率，降低了功耗。然而，要想从突发操作模式中获得最大收益，还应满足以下条件：

MIPI DSI Controller 有足够的像素 Buffer 来存储整行像素，以避免数据溢出。

显示设备应支持在单个数据包突发中，接收整行像素，避免接收缓冲区溢出。

DSI 的输出带宽应该高于 DPI 系统接口的输入带宽，使链路在每一行视频传输后，进入低功耗模式一次。

如果系统不能满足这些要求，在使用突发模式时，很可能会丢失像素数据，导致显示设备出现故障。Nor-Burst Mode 能为像素传输提供更好的速率匹配：

只有一定数量的像素存储在内存中，不需要一行完整的像素数据。

只支持少量像素缓冲（少于一整行像素）的设备进行操作。

以下计算公式给出了 MIPI DSI Controller 在 Nor-Burst Mode 下，如何匹配 DSI 链路像素输出速率和 DPI 像素输入速率。

空报文使能时：

$$\frac{\text{Lanebyteclkperiod} \times \text{vid_num_chunks} \times (\text{vid_pkt_size} \times \text{bytes_per_pixel} + 12 + \text{vid_null_size})}{\text{number_of_Lanes}} = \text{pixels_per_line} \times \text{dpipclkperiod} \quad \text{公式 4-3}$$

空报文关闭时：

$$\frac{\text{Lanebyteclkperiod} \times \text{vid_num_chunks} \times (\text{vid_pkt_size} \times \text{bytes_per_pixel} + 6)}{\text{number_of_Lanes}} = \text{pixels_per_line} \times \text{dpipclkperiod} \quad \text{公式 4-4}$$

4.5 Video Mode 插入数据包

Video Mode 下，DSI HOST 允许在视频的消隐期插入数据包。在使用时，用户需要通过寄存器 DPI_LP_CMD_TIM（0x14）告诉 DSI 模块不影响视频发送的最大可插入数据包长度，DSI 模块会以这个最大包长度为限制，将来自 APB 接口或 DPI 接口的特殊包在视频行场消隐期内发送出去。

由于数据包只能在视频的消隐期内发送，这种方式并不适用于视频前，屏幕的初始化配置，点屏常用操作流程为：将 DSI 模块配置为 Host 的 Command 模式，发送屏幕配置参数后，切回 Video 模式，发送视频数据，在视频的消隐期中插入调光等辅助数据包。

通过 APB 接口与 DPI 接口插入数据包的操作流程如下：

■ 通过 APB 接口在视频行场中插入数据包

对于短包，用户只需通过 GEN_HDR（0x58）寄存器，写入包类型与数据内容，DSI 模块会自动填充 ECC，并在视频消隐期内发送出去；

对于长包，用户需要通过 GEN_PLD_DATA（0x5c）寄存器，依次写入数据内容，同时通过 GEN_HDR（0x58）寄存器写入数据包类型与数据长度，DSI 模块会自动填充 ECC 与 CRC，在视频的行场消隐期内发送出去

需要注意的是，GEN_PLD_DATA（0x5c）寄存器有 64byte 的深度限制，用户在发送长度超过 64byte 的数据包时，需要通过 CMD_PKT_STATUS（0x68）寄存器，关注数据 Buffer 的满状态，避免数据溢出。

■ 通过 DPI 接口以特殊包的形式在视频行场中插入数据包

使用 DPI 接口以特殊包的形式在视频行场消隐中插入数据包时，需要将 MODE_CFG 寄存器的 sp_packet_en 位设置为 1，用户通过拉高 dpi_packet_en，就可以在视频的行场消隐期间插入特殊的短包或长包，DPI 接口发送特殊包的时序如下所示：

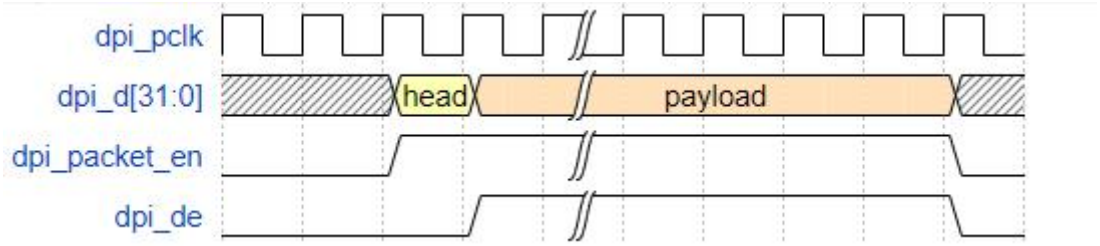

图 4-5 DPI 接口 Video Mode 特殊包传输时序图

当插入的数据包为短包时，head 代表的是数据包类型和数据内容，插入的数据包为长包时，head 代表的是数据包类型和数据个数，payload 代表的是相应长度的数据包内容。在视频的消隐期插入数据包的完整时序如下图所示，依次为 VSA 期间插入的数据包、VBP 期间插入的数据包和视频数据包。

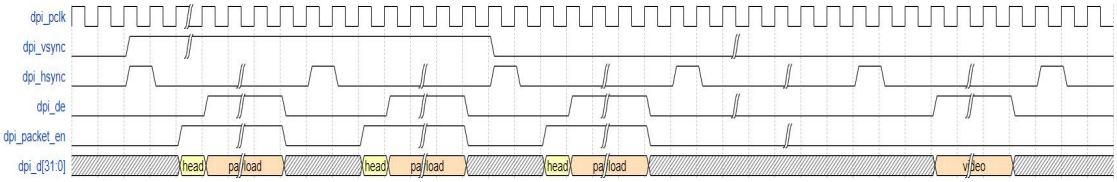

图 4-6 DPI 接口 Video Mode 包传输总时序图

以上两种数据包的插入方式，均涉及到依据数据包类型的长包、短包判断，所以 DSI 模块只能支持 DSI 协议规定类型的数据包发送。

4.6 Video Mode 测试画面

在使用 Video Mode 发送数据时，DSI 模块的 Pattern Generator 允许在没有任何激励的情况下传输水平/垂直彩条(Color Bar Pattern)或 PHY 误码率测试画面(BER Testing Pattern)。使用时，只需要在 Video Mode 发送数据的相同配置下，通过 VID_MODE_CFG（0x24）寄存器使能与选择测试图片内容，就可以将 MIPI 数据源从 DPI 接口切到测试画面。

彩条由八种颜色组成:白色、黄色、青色、绿色、品红、红色、蓝色和黑色。

通过将像素宽度(垂直模式)或像素高度(水平模式)除以 8 来计算每个颜色宽度。如果一帧中的像素宽度或者像素高度不能被 8 整除，剩余像素宽度或高度的显示和最后一种颜色(黑色)一样。

垂直模式(Vertical)Color Bar Pattern 效果如下图所示：

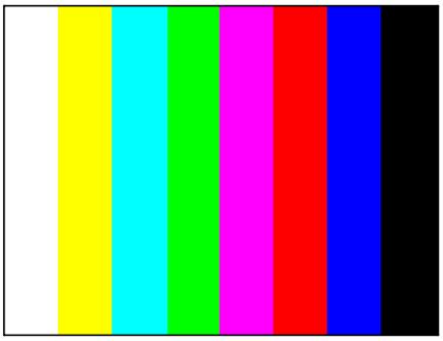

图 4-7 Vertical Color Bar Pattern

水平模式(Horizontal)Color Bar Pattern 效果如下图所示：

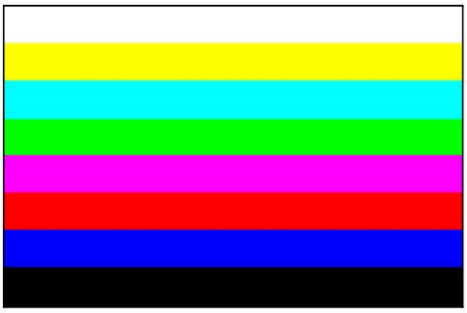

图 4-8 Horizontal Color Bar Pattern

误码率测试画面简化了一致性测试。此模式可以测试对端正确接收数据的能力。数据格式如下:

x 字节的 0xAA(高频模式，反向)

x 字节的 0x33(中频模式)

x 字节 0xF0(低频模式，反转)

x 字节的 0x7F(单独 0 模式)

x 字节的 0x55(高频模式)

x 个 0xCC 字节(中频模式，反向)

x 字节的 0x0F(低频模式)

y 字节为 0x80(单独的 1 个模式)

x 的大小为水平分辨率除以 8 的整数，Y 的大小为 x 加上水平分辨率除以 8 的余数。以 RGB888 为例，显示效果如下图所示：

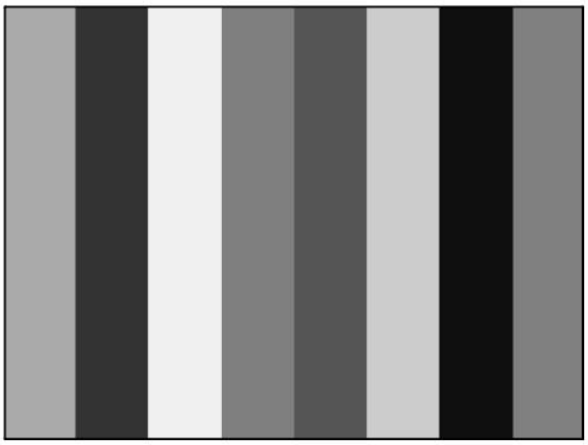

图 4-9 RGB888 BER testing Pattern

4.7 Command Mode 发送数据

通过 aIDPI 接口以命令模式(Command Mode)传输图像数据时，除了按照 DSI Host 通用寄存器配置外，还需要遵循以下步骤:

- 通过 APB 接口传输使用 DCS 命令 set_column_address 和 set_page_address，定义要刷新的图像区域。图像区域只需要定义一次，在下次修改前一直有效。
- 配置 DPI_COLOR_CODING 寄存器中的 dpi_color_coding 字段来定义像素颜色编码。
- 配置 DPI_VCID 寄存器中的 dpi_vcid 字段定义 aIDPI 生成的数据包的虚拟通道。
- 配置 EDPI_CMD_SIZE 寄存器指定每行数据长度
- 配置 MODE_CFG 寄存器使 DSI 模块工作在 Command Mode。

当 aIDPI 接口的 VSYNC 信号上升时，EDPI_CMD_SIZE 寄存器的 aldpi_allowed_cmd_size 字段的当前值被加载到内部接口函数中。在 VSYNC 信号的高电平，对输入接口的每个有效像素进行计数，当这个像素计数器首次达到 aldpi_allowed_cmd_size 时，DSI 模块准备一个 WMS 命令并发送一行数据，后面每计到 aldpi_allowed_cmd_size 时，发送一个 WMC 命令加一行数据，如果 VSYNC 在计数器达到 aldpi_allowed_cmd_size 的值之前下降，则向命令 FIFO 发出一条 Word Count(WC)值设置为与计数器值对应的 WMS 或 WMC 命令。

以 aldpi_allowed_cmd_size 的值设置为 6 个像素，每个像素 24 位为例。当 VSYNC 拉高后，控制器调度一条新的 WMS 命令，当收到 6 个像素时，一个 WMS 命令被写入命令 FIFO，在此之后，每当存储了 6 个像素时，就发送并调度一个新的 WMC，直到 VSYNC 变为低电平为止，此时，如果像素计数器不为 0，则使用有效负载 FIFO 中的剩余有效负载编写最后一个 WMC 命令。信号时序如下图所示：

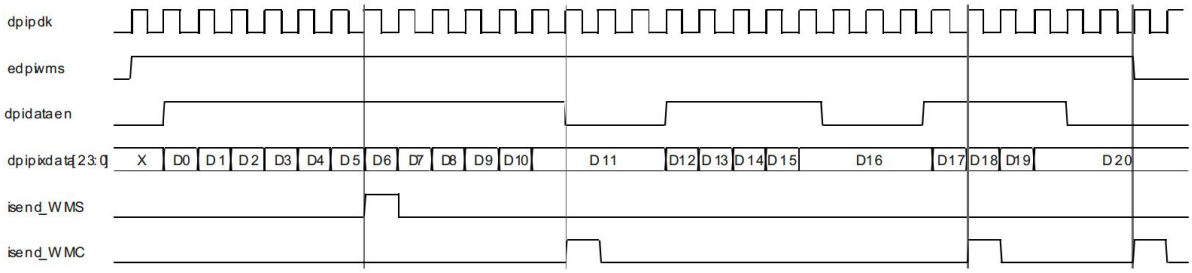

图 4-10 aIDPI 接口 Command Mode 下像素数据长包传输时序图

4.8 Command Mode 插入数据包

与 Video Mode 不同，Command Mode 下，只要总线空闲，就可以通过 APB 接口与 alDPI 接口在视频数据中插入数据包，插入数据包的操作流程如下：

■ 通过 APB 接口插入数据包

对于短包，用户只需通过 GEN_HDR（0x58）寄存器，写入包类型与数据内容，DSI 模块会自动填充 ECC，并触发一次 MIPI 传输；

对于长包，用户需要通过 GEN_PLD_DATA（0x5c）寄存器，依次写入数据内容，同时通过 GEN_HDR（0x58）寄存器写入数据包类型与数据长度，DSI 模块会自动填充 ECC 与 CRC，触发一次 MIPI 传输。

需要注意的是，GEN_PLD_DATA（0x5c）寄存器同样有 64Byte 的深度限制，用户在发送长度超过 64Byte 的数据包时，需要通过 CMD_PKT_STATUS（0x68）寄存器，关注数据 Buffer 的满状态，避免数据溢出。

■ 通过 alDPI 接口以特殊包的形式插入数据包

在使用 alDPI 接口以特殊包的形式在视频数据中插入数据包时，同样只需要将 MODE_CFG 寄存器的 sp_packet_en 位设置为 1，用户通过拉高 dpi_packet_en，就可以触发一次 MIPI 传输，当插入的数据包为短包时，Head 代表的是数据包类型和数据内容，插入的数据包为长包时，Gead 代表的是数据包类型和数据个数，Payload 代表的是相应长度的数据包内容。在视频数据中插入数据包的完整时序如下所示，图中依次为 alDPI 插入的数据包和 command Mode 下的视频数据包。

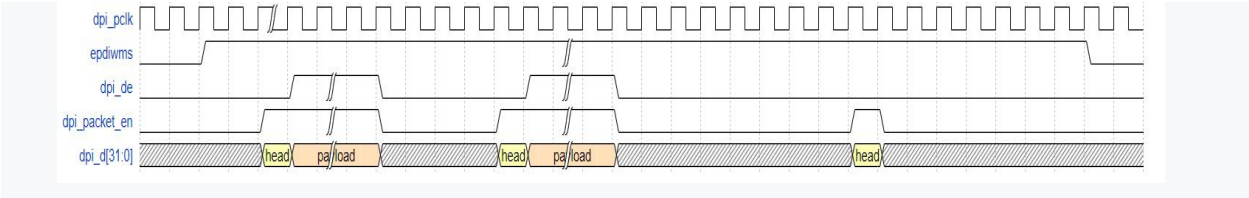

图 4-11 alDPI 接口 Command Mode 下像短包传输时序图

以上两种数据包的插入方式，均涉及到依据数据包类型的长包、短包判断，所以 DSI 模块只能支持 DSI 协议规定类型的数据包发送。

4.9 自定义数据组包发送

以上所述数据发送，都是利用 DSI Controller 的固定组包方式。DSI 模块也支持客户按照实际需求完成自定义组包发送。自定义组包的方式与 Command Mode 下插入数据包类似，都需要按照 DSI Host 寄存器配置，先将 DSI 模块配置为 Command Mode，具体实现如下：

■ 通过 APB 接口的自定义组包

APB 协议定义写和读的过程需要两个时钟周期来执行，这意味着通过 APB 接口的最大数据速率总是 APB 时钟速度的一半。当仅使用 APB 接口时，DSI 链路的理论最大比特率可以由以下公式给出：

$$\text{DSI 链路最大比特率} = \frac{\text{APB 时钟频率(MHz)}\times 32}{2}\ Mbps \qquad \text{公式 4-5}$$

公式中数字 32 代表 APB 数据总线宽度，除以 2 是因为每个 APB 读写过程需要两个时钟周期来执行。带宽依赖于 APB 时钟频率，可用带宽随时钟频率增加而增加。

在写入包数据时，首先使用 GEN_PLD_DATA 寄存器写入有效数据。然后通过 GEN_HDR 寄存器写入包头，当有效负载数据用于内存写命令(Memory Write Command)时，它包含了像素信息，它应该遵循 DCS 规范附录 a 中提到的像素到字节转换机制

■ 通过 alDPI 接口以特殊包的方式自定义组包

在使用 aIDPI 接口自定义组包时，同样只需要将 MODE_CFG 寄存器的 sp_packet_en 位设置为 1，用户通过拉高 dpi_packet_en，就可以触发一次 MIPI 传输，当插入的数据包为短包时，Head 代表的是数据包类型和数据内容，插入的数据包为长包时，Head 代表的是数据包类型和数据个数，Payload 代表的是相应长度的数据包内容。在视频数据中插入数据包的完整时序如下所示，图中依次为 aIDPI 插入的数据包和 Command Mode 下的视频数据包。

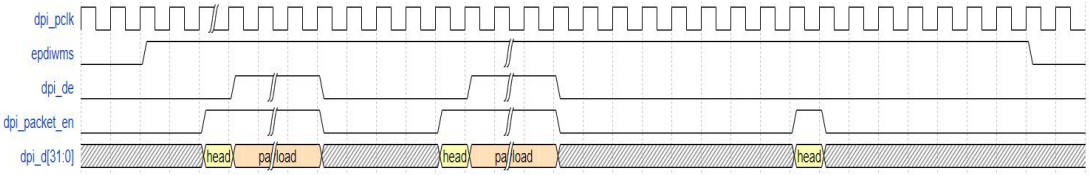

图 4-12 aIDPI 接口 Command Mode 下像短包传输时序图

APB 接口的自定义组包是通过发送长包实现的，只支持 Command Mode，aIDPI 接口的自定义组包是通过特殊包发送，涉及到依据数据包类型的长包、短包判断，支持且只支持 DSI 协议规定类型的 Command、Video 数据包自定义组合与发送。

5 附加信息

5.1 不同像素位宽在 APB 数据写总线中的数据结构。

内存写命令(Memory Write Command)以 DCS 长数据包的形式传递，DCS 长报文封装在 DSI 报文中。DSI 指定 DCS 命令应该出现在数据包的第一个有效负载字节中。在图 4-7 到图 4-11 中，可以用 DCS 的命令 Write Memory Start(WMS)和 Write Memory Continue(WMC)来替换内存写命令(Memory Write Command)。

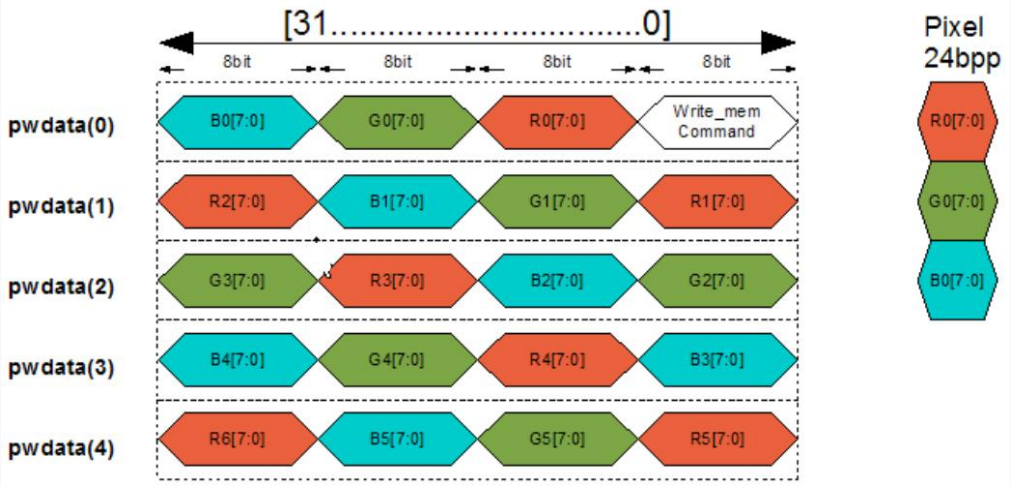

图 5-1 24 bpp RGB 像素 APB 数据总线结构图

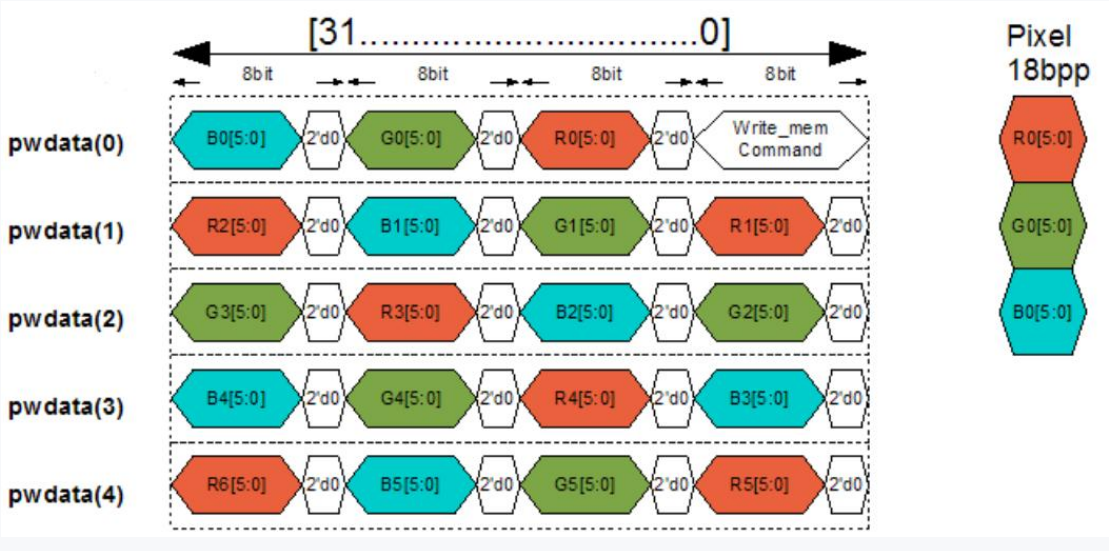

图 5-2 18 bpp RGB 像素 APB 数据总线结构图

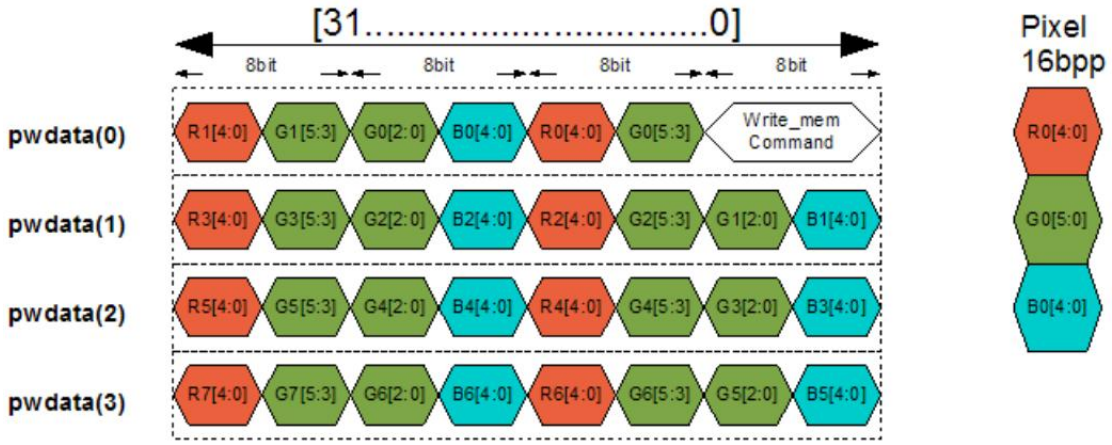

图 5-3 16 bpp RGB 像素 APB 数据总线结构图

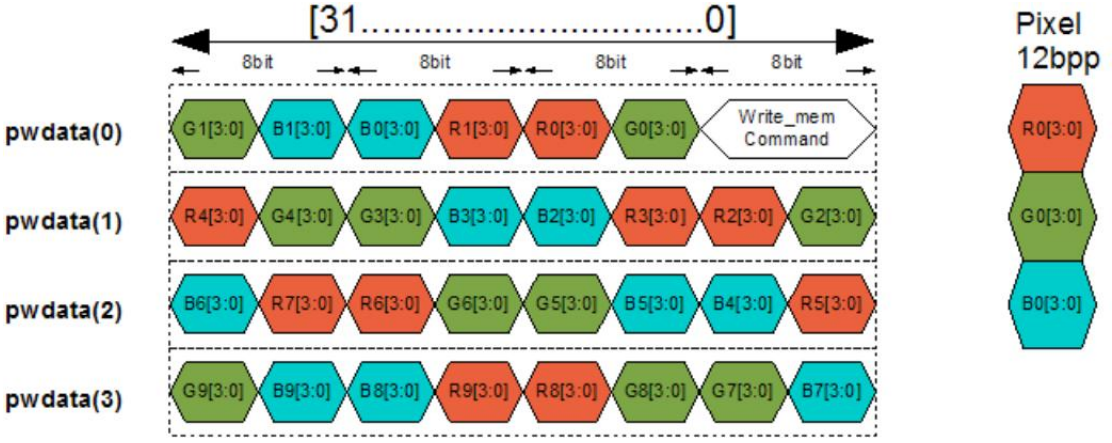

图 5-4 12 bpp RGB 像素 APB 数据总线结构图

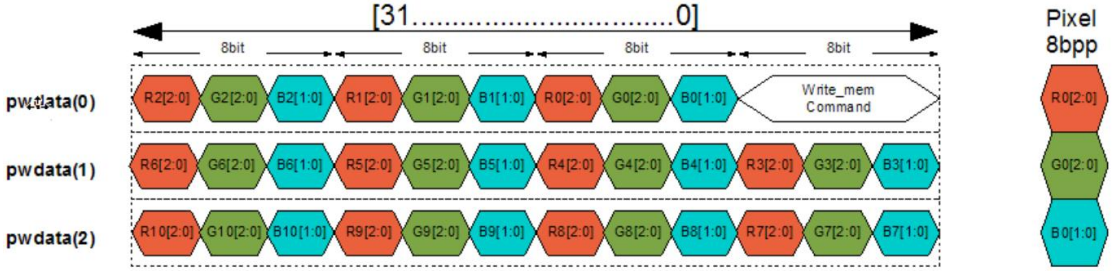

图 5-5 8 bpp RGB 像素 APB 数据总线结构图

5.2 DPI/aIDPI 接口颜色模式数据映射

表 5-1 DPI/aIDPI 接口颜色模式数据映射表

Color Mode mapping	16-Bit			18-Bit		24-Bit		24-Bit YCbCr 4:2:2	
	Config1	Config2	Config3	Config1	Config2			1st Cycle	2nd Cycle
host_dpi_d[31]									
host_dpi_d[30]									
host_dpi_d[29]							R[9]		
host_dpi_d[28]							R[8]		

host_dpi_d[27]							R[7]		
host_dpi_d[26]							R[6]		
host_dpi_d[25]							R[5]		
host_dpi_d[24]							R[4]		
host_dpi_d[23]						R[7]	R[3]	Y1[11]	Y2[11]
host_dpi_d[22]						R[6]	R[2]	Y1[10]	Y2[10]
host_dpi_d[21]			R[4]		R[5]	R[5]	R[1]	Y1[9]	Y2[9]
host_dpi_d[20]		R[4]	R[3]		R[4]	R[4]	R[0]	Y1[8]	Y2[8]
host_dpi_d[19]		R[3]	R[2]		R[3]	R[3]	G[9]	Y1[7]	Y2[7]
host_dpi_d[18]		R[2]	R[1]		R[2]	R[2]	G[8]	Y1[6]	Y2[6]
host_dpi_d[17]		R[1]	R[0]	R[5]	R[1]	R[1]	G[7]	Y1[5]	Y2[5]
host_dpi_d[16]		R[0]		R[4]	R[0]	R[0]	G[6]	Y1[4]	Y2[4]
host_dpi_d[15]	R[4]			R[3]		G[7]	G[5]	Y1[3]	Y2[3]
host_dpi_d[14]	R[3]			R[2]		G[6]	G[4]	Y1[2]	Y2[2]
host_dpi_d[13]	R[2]	G[5]	G[5]	R[1]	G[5]	G[5]	G[3]	Y1[1]	Y2[1]
host_dpi_d[12]	R[1]	G[4]	G[4]	R[0]	G[4]	G[4]	G[2]	Y1[0]	Y2[0]
host_dpi_d[11]	R[0]	G[3]	G[3]	G[5]	G[3]	G[3]	G[1]	Cb1[11]	Cr2[11]
host_dpi_d[10]	G[5]	G[2]	G[2]	G[4]	G[2]	G[2]	G[0]	Cb1[10]	Cr2[10]
host_dpi_d[9]	G[4]	G[1]	G[1]	G[3]	G[1]	G[1]	B[9]	Cb1[9]	Cr2[9]
host_dpi_d[8]	G[3]	G[0]	G[0]	G[2]	G[0]	G[0]	B[8]	Cb1[8]	Cr2[8]
host_dpi_d[7]	G[2]			G[1]		B[7]	B[7]	Cb1[7]	Cr2[7]
host_dpi_d[6]	G[1]			G[0]		B[6]	B[6]	Cb1[6]	Cr2[6]
host_dpi_d[5]	G[0]		B[4]	B[5]	B[5]	B[5]	B[5]	Cb1[5]	Cr2[5]
host_dpi_d[4]	B[4]	B[4]	B[3]	B[4]	B[4]	B[4]	B[4]	Cb1[4]	Cr2[4]
host_dpi_d[3]	B[3]	B[3]	B[2]	B[3]	B[3]	B[3]	B[3]	Cb1[3]	Cr2[3]
host_dpi_d[2]	B[2]	B[2]	B[1]	B[2]	B[2]	B[2]	B[2]	Cb1[2]	Cr2[2]
host_dpi_d[1]	B[1]	B[1]	B[0]	B[1]	B[1]	B[1]	B[1]	Cb1[1]	Cr2[1]
host_dpi_d[0]	B[0]	B[0]		B[0]	B[0]	B[0]	B[0]	Cb1[0]	Cr2[0]

版本信息

日期	版本	说明
2022/4/15	1.0	初版建立

版权所有©2022 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其它方式授予任何知识产权许可。除安路科技在其产品的销售条款和条件中声明的责任之外，安路科技概不承担任何法律或非法律责任。安路科技对安路科技产品的销售和/或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。安路科技对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，安路科技保留修改文档中任何内容的权利，恕不另行通知。安路科技不承诺对这些文档进行适时的更新。